UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

LABORATORIO SISTEMAS OPERATIVOS 1

Manual Tecnico

Nombre	Carnet
Christopher Miguel Angel Ramos Ascencio	202200057

Introducción

Este proyecto implementa un sistema de telemetría y gestión autónoma de contenedores Docker, combinando tres niveles de observabilidad del sistema operativo:

1. Un **módulo de kernel en C** que expone métricas de RAM y de todos los procesos del sistema a través de `/proc`.
2. Una **sonda eBPF** que audita en tiempo real las señales de terminación (`kill`/`pidfd_send_signal`) que el sistema envía a los procesos.
3. Un **daemon en Go** que actúa como orquestador: lee las métricas del kernel, decide qué contenedores mantener y cuáles eliminar según reglas de consumo, ejecuta las eliminaciones, y usa la sonda eBPF para confirmar que cada eliminación realmente ocurrió — antes de registrarla en Valkey.

Todo el flujo se visualiza en un dashboard de Grafana en tiempo real.

Flujo de una eliminación de contenedor:

1. El daemon lee `/proc/continfo_pr2_so1_202200057` cada 20-30s.
2. Decide qué contenedores sobran (ver sección 5.2).
3. Le pide a Docker que envíe `SIGTERM` (y si no muere, `SIGKILL`) al contenedor.
4. Espera, consultando `docker inspect`, a que el contenedor realmente deje de estar corriendo.
5. En paralelo, la sonda eBPF captura la señal real que el kernel entregó.
6. Solo cuando **ambas** condiciones se cumplen (Docker confirma la muerte Y eBPF capturó la señal), el daemon escribe "contenedor eliminado" en Valkey e incrementa el contador de eventos eBPF.

Módulo de kernel

Qué expone

`/proc/continfo_pr2_so1_202200057` contiene:

- RAM total, libre y en uso del sistema (`si_meminfo()`).
- Una tabla con **todos** los procesos del sistema: PID, nombre, línea de comandos, VSZ, RSS, %MEM y %CPU.
-

Decisiones de diseño

Dos pasadas sobre la lista de procesos. La primera pasada recorre `for_each_process()` bajo `rcu_read_lock()` únicamente para recolectar punteros (`get_task_struct()` sobre cada uno), porque ese contexto no permite bloquear. La

segunda pasada, ya fuera del lock, puede usar funciones que sí bloquean (`get_task_mm()`, acceso a memoria de otro proceso) sin arriesgar un deadlock.

Lectura del cmdline sin `get_cmdline()`. La función interna del kernel `get_cmdline()` no está exportada para módulos externos en la versión de kernel de esta máquina (`7.0.0-31-generic`) — intentar usarla produce un error de enlazado (`undefined symbol`) al cargar el módulo. En su lugar, el módulo lee `mm->arg_start`/`arg_end` del proceso objetivo y usa `access_process_vm()` (sí exportada de forma estable) para copiar los argumentos directamente desde el espacio de memoria de ese proceso.

Estimación de %CPU. Se calcula como una relación diferencial simple entre `task->utime + task->stime` y `jiffies - task->start_time`. El enunciado permite explícitamente que este número salga grande o poco intuitivo, ya que el kernel entrega esos tiempos en unidades distintas a jiffies.

eBPF

Diseño inicial

La sonda se adjunta al tracepoint `syscalls:sys_enter_kill` y, cada vez que se invoca la syscall `kill()`, envía un evento (PID emisor, PID receptor, señal) al daemon de Go a través de un ring buffer.

Docker no usa `kill()`

Durante las pruebas, el daemon nunca lograba confirmar sus propias eliminaciones — la sonda capturaba señales de fondo del sistema, pero nunca las dirigidas a los contenedores que el daemon mataba. Diagnosticando con `bpftrace`, escuchando simultáneamente `sys_enter_kill` y `sys_enter_pidfd_send_signal`, se confirmó que **Docker/containerd en esta versión usa `pidfd_send_signal()`**, no `kill()` — una syscall más moderna que evita condiciones de carrera con PIDs reciclados.

Solución: se agregó una segunda sonda para `syscalls:sys_enter_pidfd_send_signal`. Esta syscall no recibe el PID objetivo directamente, sino un *pidfd* (file descriptor que apunta al proceso). Resolver ese pidfd al PID real se hace en espacio de usuario (no en el programa eBPF, para mantenerlo simple): el daemon de Go lee `/proc/<pid_emisor>/fdinfo/<pidfd>`, donde el kernel expone una línea `Pid:\t<pid_real>` para cualquier pidfd abierto (disponible desde Linux 5.13+).

Daemon de Go

Ciclo de vida

1. **Inicio:** levanta Grafana y Valkey vía *`docker compose up -d`.*
2. **Cronjob:** instala la generación periódica de contenedores (`crontab`, cada 2 minutos).
3. **Módulo de kernel:** ejecuta el script que compila y carga `continfo.ko`.
4. **Sonda eBPF:** se adjunta a ambos tracepoints (sección 4).
5. **Loop principal:** cada 20-30s, lee `/proc`, decide y elimina.
6. **Finalización:** al recibir `SIGINT`/`SIGTERM`, elimina el cronjob antes de salir.

Lógica de decisión

El cronjob (*`cronjob_generate_containers.sh`*) despliega 5 contenedores elegidos **aleatoriamente** entre los 3 perfiles de carga, más el contenedor intruso — tal como exige el enunciado ("Despliegue Aleatorio").

Mantener el equilibrio de "siempre 3 de bajo consumo y 2 de alto consumo" es responsabilidad exclusiva del **daemon**, no del cronjob. Cada contenedor se crea con una label de Docker (`proyecto2_profile=low` o `high`), que es la fuente de verdad para clasificarlo — se prefirió esto sobre medir %CPU/%MEM en vivo, porque esas métricas pueden salir ruidosas (se observaron picos de cientos de % en procesos del sistema).

En cada iteración:

1. El contenedor intruso, si existe, se marca para eliminar sin condición.
2. Los demás se agrupan por su label de perfil.
3. Dentro de cada grupo, se ordenan por consumo real (%MEM + %CPU) de mayor a menor.
4. Se eliminan los que sobran de cada grupo hasta dejar exactamente 3 de bajo consumo y 2 de alto consumo — ni más overhead del necesario, ni menos del mínimo exigido.

Envío de la señal de terminación

La primera versión enviaba la señal directamente desde el propio proceso del daemon (`syscall.Kill`), pero esto fallaba con `permission denied` en esta máquina: el perfil AppArmor bajo el que corre la terminal (heredado del editor) bloqueaba el envío directo de señales a procesos fuera de su confinamiento. La solución fue delegar el envío a `docker kill --signal=<SIGTERM|SIGKILL>`, que sí está autorizado por el perfil `docker-default` de AppArmor.

Confirmación de eliminación (dos condiciones)

Un contenedor solo se registra como "eliminado" en Valkey cuando:

1. *`docker inspect`* confirma que el contenedor ya no está corriendo, **y**
2. la sonda eBPF observó una señal **`SIGTERM`/`SIGKILL`** durante la ventana de terminación.

Si el contenedor muere pero no se observa la señal correspondiente en la ventana de tiempo esperada, el daemon lo deja explícito en el log como advertencia en vez de registrar una confirmación falsa — se prefirió fallar de forma visible a mentir en los datos.

Dashboard de Grafana

El dashboard se provisiona automáticamente desde `grafana/provisioning/dashboards/proyecto2_dashboard.json`, montado como volumen en el contenedor de Grafana. Esto significa que el dashboard **sobrevive** a que se borre el volumen de datos de Grafana (`docker compose down -v`) — se reconstruye solo al arrancar, en vez de depender de que alguien lo vuelva a armar a mano en la interfaz.

Paneles:

- **Total RAM / Free RAM:** tarjetas, *`GET system:ram:latest`*.
- **RAM en uso:** gauge, mismo origen de datos.
- **Total contenedores eliminados:** tarjeta, *`GET removed:total_count`*.
- **Top 5 por RAM / CPU:** pie charts, *`ZREVRANGE ranking:ram|cpu`*.
- **Contador de eventos eBPF:** tarjeta, *`GET ebpf:events_total`*
  donde por definición del enunciado, cuenta los contenedores eliminados confirmados vía eBPF (por eso coincide con "Total contenedores eliminados").

Guía de instalación paso a paso

bash

1. Dependencias (Linux Mint / Ubuntu 24.04)

**sudo apt install -y build-essential linux-headers-$(uname -r) **

**docker-compose-v2 clang llvm libbpf-dev libelf-dev **

linux-tools-common linux-tools-generic valkey-tools bpftrace

2. Módulo de kernel

cd kernel

make CARNET=202200057

sudo insmod continfo.ko

3. Imágenes de carga

docker pull roldyoran/go-client

docker pull alpine

docker build -t proyecto2-intruder:latest docker/images/intruder

4. Infraestructura

docker compose up -d

5. Sonda eBPF

cd ebpf && make

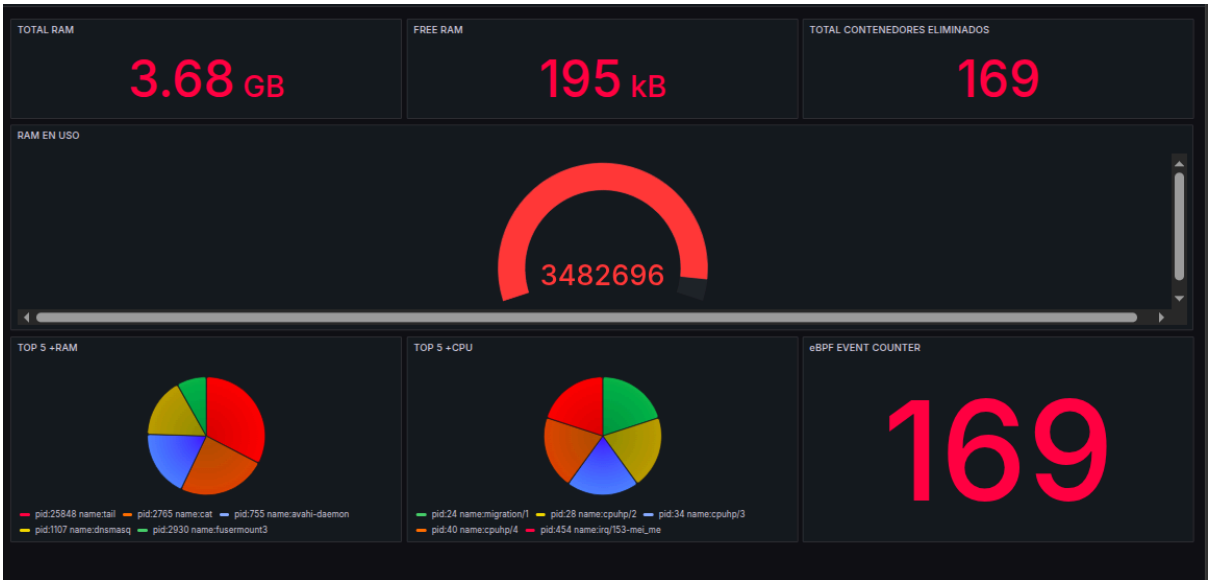
6. Daemon (orquesta todo lo anterior automáticamente al arrancar)

cd ../daemon

go build -o proyecto2daemon .

sudo ./proyecto2daemon

Evidencia funcional



9. Decisiones y aprendizajes

- **`get_cmdline()` → `access_process_vm()`:** función no exportada en esta versión de kernel, se implementó la lectura de cmdline manualmente.
- **Clasificación por label, no por métrica en vivo:** las métricas de CPU del módulo de kernel pueden ser ruidosas; la label de Docker puesta al crear el contenedor es una fuente de verdad determinística.
- **`docker kill` en vez de syscall directa:** AppArmor bloqueaba el envío directo de señales desde el daemon; delegarlo a Docker evita el problema y es igualmente válido, ya que Docker termina llamando a la syscall real.
- **`pidfd_send_signal()` además de `kill()`:** el hallazgo más importante del proyecto — Docker moderno no usa la syscall que el enunciado sugiere como ejemplo (`sys_kill`), sino su sucesora. Diagnosticado con `bpftrace` antes de tocar código, para confirmar la causa antes de intentar un arreglo a ciegas.

Diccionario de Llaves (Key Dictionary):

Llave (Key)	Tipo de Dato (Valkey)	Descripción y Propósito
system:ram:latest	**String (JSON)**	Almacena el último snapshot de la memoria (Total y Libre). Se sobrescribe constantemente. Grafana lo lee vía JSON Path ($.total_ram_mb).
system:ram:timeline	**Sorted Set (ZSET)**	Serie temporal. El **Score** contiene el Timestamp (Milisegundos) y el **Member** contiene el valor en MB. Utilizado para la gráfica de línea de tiempo.
ranking:ram / ranking:cpu	**Sorted Set (ZSET)**	Rankings para gráficas circulares. El **Score** guarda el porcentaje/cantidad, y el **Member** el PID+Nombre. Consultados con ZREVRANGE para obtener el Top 5.
ebpf:events_total	**String (Contador)**	Valor entero crudo. Incrementado atómicamente por la goroutine que lee el Ring Buffer de eBPF por cada señal de kernel capturada.
removed:total_count	**String (Contador)**	Incrementado **solo** cuando la función de cruce eBPF confirma que un contenedor objetivo fue cerrado mediante SIGTERM/SIGKILL.